HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 07

MCP-Enabled Workflows & Custom MCP Server Creation

Give your agents safe, governed access to repositories, docs and enterprise systems. Learn what MCP is, integrate it in Copilot, run a real workflow, then build and govern a custom MCP server.



DURATION

3 hours



FORMAT

Concepts · 2 labs



PRIMARY TOOL

Copilot agent mode + MCP

How We'll Spend These Three Hours

From the concept, to integration, to a working workflow, to building and governing your own server.

01

MCP Concepts & Architecture

25 min

The problem MCP solves, hosts/clients/servers, tools/resources/prompts, and one tool call end to end.

02

Integrating MCP in Copilot

25 min

Configuring servers in VS Code, trust and policy, and a gallery of real-world integrations.

03

Lab 1: Repo to Spec Validation

35 min

An MCP-enabled workflow: repository analysis → specification validation → result analysis.

04

Designing a Custom MCP Server

30 min

When to build, tool design principles, and a minimal server you can run in minutes.

05

Lab 2: Build, Wire & Govern

45 min

Stand up a server with one or two tools, wire it into an agent workflow, document its governance.

06

Security, Governance & Best Practices

20 min

Threats and controls, approvals, least privilege, and what to document for every server.

Module 07 in the 12-Module Journey

The first module of the EXTEND phase: agents stop being limited to what is in the editor.

FOUNDATION			BUILD & VALIDATE			EXTEND WITH TOOLS & AGENTS				GOVERN	CAPSTONE
1	2	3	4	5	6	**7**	8	9	10	11	12
Objectives & GenAI Recap	Copilot Advanced	Prompt & Context Eng.	Spec-Driven Dev (Spec Kit)	SDD to Full SDLC	Test Strategy & Validation	**MCP & Custom MCP Server**	PR Automation & LLM-Judge	No-Code Agents (Rovo)	Code Agents & Simple RAG	Observability, ROI & Gov.	Capstone & Roadmap

M03
Tool results are context — select, compress and scope them

M04–06
Specs and tests are what an MCP workflow validates against

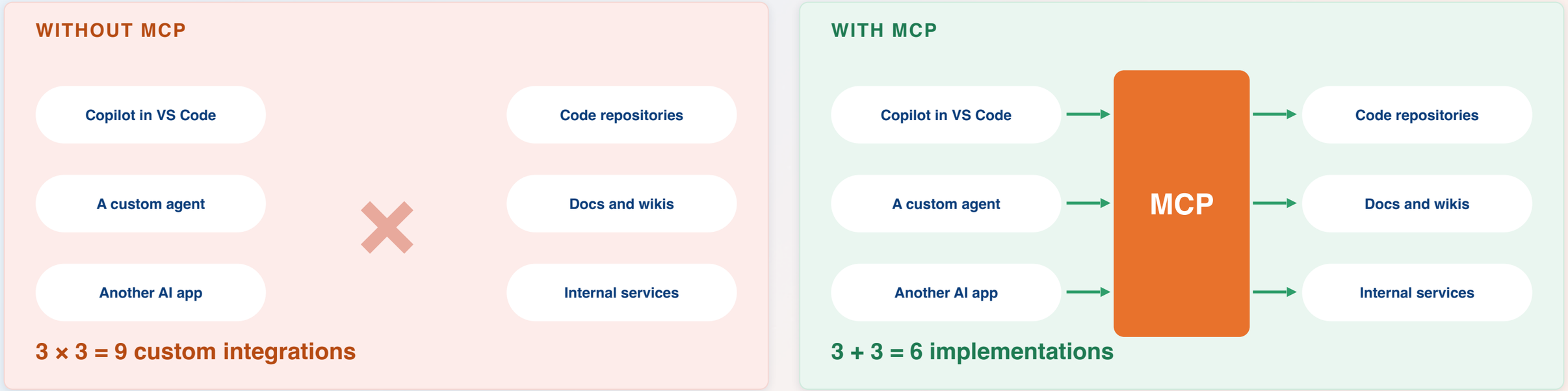
M08
PR review agents use MCP tools and need the same governance

M10
Code agents and RAG reuse MCP servers as their tools

Why this matters: an agent is only as useful as what it can reach, and only as safe as the limits you put on that reach. MCP is the standard way to do both.

The Problem MCP Solves: One Protocol Instead of N × M Integrations

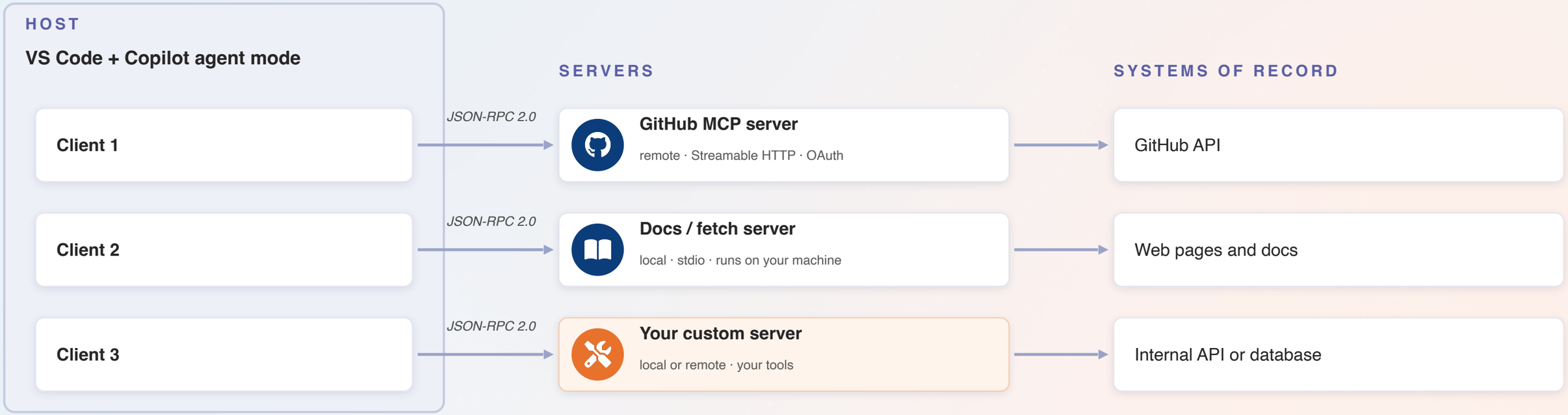
Every AI app connecting to every tool used to mean custom glue for each pair.



Think of it as: a common plug for AI apps and tools. Build or approve a server once; any MCP-capable host can use it. The numbers show the counting argument for 3 apps and 3 tools — real estates have far more.

MCP Architecture: Host → Client → Server → System

One client per server inside the host. Servers can run on your machine or remotely.



Two transports: stdio for local servers (a process on your machine) and Streamable HTTP for remote servers (reachable by URL, usually with OAuth). The protocol on top is the same for both.

What a Server Offers: Tools, Resources and Prompts

Three building blocks a server can expose — each with a different controller.

TOOLS

Actions the model can call

Model-controlled

Examples (illustrative)

```
create_issue(title, body)
get_workflow_run(id)
search_code(query)
```

RESOURCES

Data the app can read as context

Application-controlled

Examples (illustrative)

```
file:///repo/README.md
db://schema/orders
docs://api/v2/overview
```

PROMPTS

Reusable templates a user can invoke

User-controlled

Examples (illustrative)

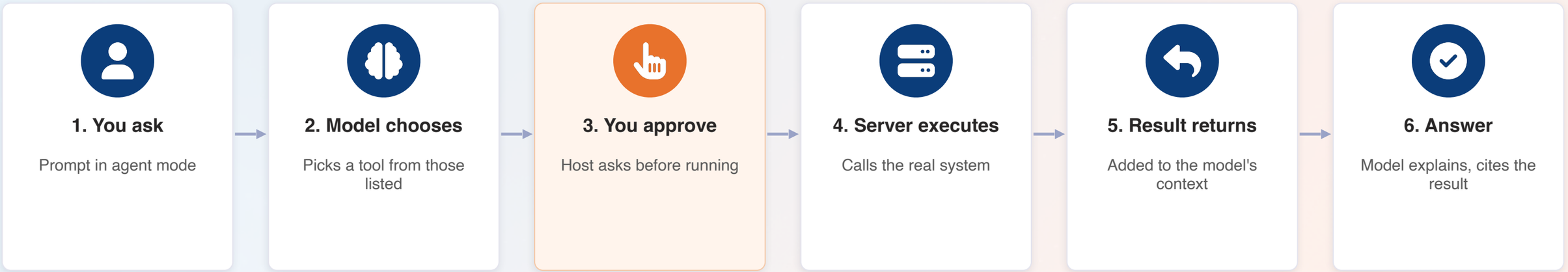
```
/review-pr
/explain-failure
/draft-release-notes
```

Client feature — elicitation: a server can ask the user for missing input mid-task, instead of guessing. Hosts differ in which features they support.

Rule of thumb: if the model should decide to do it, make it a tool. If it is just context, make it a resource. If a human triggers it, make it a prompt.

One Tool Call, End to End

What actually happens when you ask the agent: “Why did the last build fail?”



Tool results are untrusted input: step 5 puts text from an external system into the model's context.

Why this matters: step 3 is your control point and step 5 is your risk point. Everything in the security slide comes back to these two.

Integrating MCP in VS Code and Copilot

Add a server, trust it once, then choose which of its tools the agent may use.

WORKSPACE CONFIG — SHARED WITH THE TEAM

.vscode/mcp.json (illustrative)

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "team-docs": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "<docs-server-package>"]
    }
  }
}
```

Add the server

Edit mcp.json or use the MCP commands in the Command Palette.

Confirm trust

VS Code asks you to trust a server before it starts — read the command first.

Pick the tools

Open the tools picker in Chat; enable only what the task needs.

Check org policy

Organisations can control MCP use in Copilot; it is off by default for some plans.

Never commit secrets: use input variables or environment files for tokens in mcp.json. The file is shared; the credentials must not be.

Real-World Integrations: What Teams Connect First

Start with read-heavy, low-risk servers; add write access once the workflow is proven.

GitHub: issues and PRs

Read + write

Summarise, triage, draft and link issues and pull requests from the editor.

Workflow failure analysis

Read-only

Fetch a failed CI run's logs and ask the agent for likely causes.

Dependency alerts

Read-only

List open Dependabot alerts and group them by severity and package.

Repository analysis

Read-only

Search code and structure so the agent answers from your repo, not guesses.

Docs and knowledge

Read-only

Expose internal docs and API references as resources the agent can read.

Internal services

Custom server

Wrap a ticketing, test-lab or build API as a few well-scoped tools.

Tip: the GitHub MCP server supports a read-only mode and toolsets, so you can expose just issues, or just PRs, instead of everything.

An MCP-Enabled Workflow: Repo → Spec Check → Result Analysis

Three agent steps, each using different tools, with a human checkpoint before anything is written.

1 Repository analysis

Agent finds the code and open issues related to the spec's requirements.

TOOLS USED (illustrative)

`search_code`

`read_file`

`list_issues`

2 Specification validation

Agent checks each acceptance criterion against code and test results; marks met / not met / unknown.

TOOLS USED (illustrative)

`spec.md (resource)`

`get_workflow_run`

`read_file`

3 Result analysis

Agent summarises gaps and drafts follow-up issues. You review before anything is posted.

TOOLS USED (illustrative)

`create_issue (draft)`

`add_comment (draft)`

Design choice: steps 1–2 are read-only. The only write is a draft in step 3, and a human approves it. Prove the workflow read-only first, then add writes.

Lab 1: Connect and Validate

LAB 1 · 35 min

Wire the GitHub MCP server into Copilot and run the three-step workflow on your SDD feature.

Goal Connect a read-only MCP server, then use agent mode to check your feature's spec against the repo and produce a gap list.

STEPS

1. Add the GitHub MCP server in .vscode/mcp.json, trust it, and enable only the read tools you need in the tools picker.
2. Prompt: find the code and tests for each acceptance criterion in spec.md. Watch which tools it calls and approve each one.
3. Ask it to mark each criterion met / not met / unknown, citing the file or run that supports it.
4. Have it draft (not post) one follow-up issue for a gap. Review the draft and edit it.

FILES YOU CREATE

repo/

```
lab07a/
  .vscode/mcp.json
  spec.md
  validation-report.md
  draft-issue.md
```

DONE WHEN

validation-report.md lists every criterion with a status and evidence, and you can name every tool call the agent made.

Building Your Own: When, and How to Design the Tools

Build a server when an approved one does not exist for your system — and keep it small.

BUILD OR REUSE?

Is there an approved server for it?	→ Reuse it
Does it need only a few operations?	→ Build small
Does it touch production data?	→ Review first

TOOL DESIGN PRINCIPLES

- **One tool, one job —** Clear name, narrow purpose
- **Describe it for the model —** The description is the instruction
- **Typed, validated inputs —** Reject bad arguments early
- **Small, structured output —** Return what is needed, not a dump
- **Safe by default —** Read-only unless writes are essential

Anti-pattern: a single run_query(sql) or run_command(cmd) tool. It hands the model your whole system. Expose the three questions you actually need answered.

A Minimal Server in About Twenty Lines — Then Test It

Implement with an SDK (here, FastMCP for Python), then verify with the MCP Inspector before any agent uses it.

server.py (illustrative)

```python
from fastmcp import FastMCP

mcp = FastMCP("Build Tools")

@mcp.tool
def get_build_status(build_id: str) -> dict:
    """Return status and failing step for a build.
    Read-only. build_id looks like 'b-1234'."""
    if not build_id.startswith("b-"):
        raise ValueError("invalid build id")
    # call your real build API here
    return {"id": build_id, "status": "failed",
            "step": "unit-tests"}

if __name__ == "__main__":  mcp.run()
```

Run it

Start locally with the SDK's dev command and open the Inspector.

Call each tool

Valid input, invalid input, missing input. Check output shape.

Check errors

Failures should be clear messages, not stack traces or secrets.

Register it

Add to .vscode/mcp.json (stdio) and try it in agent mode.

Verify against current docs: SDK syntax and CLI commands change between versions. Treat this as a shape, not a copy-paste recipe.

Lab 2: Build, Wire, Govern

LAB 2 · 45 min

Create a small server for a system your team uses, then use it in an agent workflow.

Goal Build a server with one or two read-only tools, test it, wire it into Copilot, and document how it is governed.

STEPS

1. Pick a system (build status, test results, docs index). Write the tool names, inputs and outputs on paper before any code.
2. Implement one or two tools with Copilot's help. Validate inputs and keep outputs small. Test in the Inspector.
3. Register it in .vscode/mcp.json and run a short agent prompt that needs your tool. Note what you had to approve.
4. Fill in governance.md: owner, purpose, tools, data touched, auth, approval rule, how to disable.

FILES YOU CREATE

```
repo/
lab07b/
  server.py
  .vscode/mcp.json
  governance.md
  test-notes.md
```

DONE WHEN

Your tool works in agent mode, handles one bad input cleanly, and governance.md is complete enough for a reviewer to approve or reject.

Security and Governance: Threats and Their Controls

Each risk has a practical control. Most of them live at the approval and result steps you saw earlier.

RISK	CONTROL
Prompt injection via tool results	Treat results as untrusted; review before acting; avoid auto-approve on write tools
Over-broad tools and permissions	Least privilege; read-only mode; enable only needed toolsets
Untrusted or malicious servers	Allow-list approved servers; inspect the command before trusting
Leaked credentials	Scoped tokens, input variables, never in committed config
No audit trail	Named owner, logging, and a documented way to disable a server

Policy note: organisations can restrict MCP in Copilot, and availability depends on plan. Ask your admin before rolling a server out beyond a pilot.

Governing a Server: The One-Page Record

If a reviewer cannot answer these seven questions from one page, the server is not ready.

Owner	Who maintains and fixes it?
Purpose	What job does it do? What is out of scope?
Tools	Each tool: inputs, outputs, read or write
Data	What systems and data classes can it reach?
Auth	How does it authenticate? Which scopes?
Approval	Which calls need a human yes?
Off switch	How is it disabled quickly?

Habit: store governance.md next to the server code and review it in the same pull request as any new tool.

Best Practices for MCP Workflows and Servers

Start narrow, keep a human at the checkpoints, and treat every server as production software.

DO THIS	AVOID THIS
Reuse an approved server before building your own	Install servers from unknown sources without reading them
Start read-only; add writes only with approval steps	Auto-approve write or delete tools
Enable only the tools the current task needs	Expose raw SQL or shell as a single tool
Write tool descriptions and typed inputs for the model	Put tokens or secrets in mcp.json or logs
Test with the Inspector, including bad input	Return huge payloads that flood the context
Keep a one-page governance record per server	Assume a server is safe because the protocol is standard

Golden rule: MCP standardises how an agent connects. It does not decide what the agent should be allowed to do — you do.

What You Can Do Now

Six outcomes from today — from understanding MCP to shipping a governed server.

- ✓ Explain hosts, clients, servers and the tools / resources / prompts model
- ✓ Configure MCP servers in VS Code and choose tools deliberately
- ✓ Run a repo → spec validation → result analysis workflow
- ✓ Design and build a small, read-only custom MCP server
- ✓ Test a server with the Inspector and register it for agent use
- ✓ Apply security controls and document governance for every server

NEXT: MODULE 08 — PR AUTOMATION, QUALITY GATES & LLM-AS-JUDGE · 1 HOUR

Use agents and MCP tools to review pull requests, enforce quality gates and judge AI output.

TODAY'S PATH

Concepts → Configure → Integrate → Build → Govern

Questions & Discussion

Module 07 — MCP-Enabled Workflows & Custom MCP Server Creation